The Last Mile Is Located, Not Crossed: A Kernel-Verified System That Names What It Lacks

Karl Olivet*

Preprint — August 2026

Abstract

A proof search that fails usually returns nothing an engineer can use. We report a system in which failure returns *the list of formulas that would close the goal if added to the pool* — an object, not a status. Around that primitive, twenty-one search organs accumulated over four months, and the way they accumulated is this paper's main claim: **not one came from an architect's intuition**. Each answers a replayable failure, and the catalogue's most informative column is not what an organ does but which diagnosis produced it. The trajectory that emerges was not planned. Up to a point the system *chains* what it is given; then it accepts *proposed* witnesses; then it *fabricates* one, using Bourbaki's τ to build a canonical term from the goal alone — and the move was forced by a gap the machine itself had written, $\neg(n = n)$. We also report what the instrument buys elsewhere: given an operation invented the same morning, $a \oplus b := (a + b) + 1$, and two raw laws about $+$, three organs establish its commutativity in under two seconds and its associativity in 16.43 s, kernel-closed. Two negative results are the practical payoff. A search that fails *by growing* — cost doubling per level, gap count frozen — is almost never repaired by more budget; twice out of twice the cause was upstream. And a corpus of measured traps, fourteen of them, includes a structural law of this kernel that no amount of design could have anticipated: *terms are opaque, formulas decompose*. Finally, and least comfortably, we report that our own documentation of this work had drifted on eight figures out of eight, and what that says about a paper whose thesis is that one measures rather than decrees.

1 Introduction

Ask an automated prover for a theorem it cannot reach and it returns failure: a timeout, an exhausted budget, an empty list. The information content of that answer is close to zero, and it is the same answer whether the goal was one lemma

*Independent researcher. Contact: `karl.olivet@gmail.com`. Code and certified corpus: `https://github.com/Karl-O/bourbaki-theorie-des-ensembles`, at the tag `preprint-v1` (`54ceef5`), which pins the exact state every measurement reported here was taken on.

away or unreachable in principle. Everything the search learned on the way — which subgoals closed, which hypothesis refused to discharge, which branch died and why — is discarded with it.

This paper reports a system built on the opposite convention. When its search fails, it returns **the set of formulas that would close the goal if they were added to the pool**. Failure becomes a datum with a shape: not "no", but "no, and here is exactly what is missing". We call the component that does this the *need organ*.

The last mile, defined. We borrow the term from logistics, where it names the final stretch of a delivery — short, disproportionately expensive, and the part no amount of investment in the trunk network shortens. In proof search it names this:

> Given a goal, a search derives what it can and stops. **The last mile is the distance between what it derived and what was asked** — the stretch where mechanised search has done everything it knows how to do and the goal is still open.

Three properties make it its own problem rather than merely "the hard part". It is *not proportional to effort*: doubling the budget does not halve it, because what is usually missing is not more search but a different object — a witness the pool never named, a law at an instance nobody generated. It is *not visible*: a timeout looks identical whether one lemma or a whole theory is absent. And it is *where the human comes back in*, which is exactly why leaving it undescribed is expensive — the person who resumes the work has to rediscover what the machine already knew and discarded.

The thesis. The last mile is not crossed in one jump; it is *located*. A system that fails while producing the exact statement of its lack is worth more than one that succeeds half the time without knowing why — because the first can be improved by construction and the second can only be retried.

What makes the claim testable. A thesis about how tools should grow is easy to assert and hard to check. Ours is checkable because the growth is on record. The organ accreted twenty-one extensions over four months, and for each one the repository holds the failure that motivated it, in a form that still replays. The catalogue in Section 4 therefore carries two columns, and **the right one — "born from which diagnosis" — is the load-bearing one**. If the thesis were false, that column would read like a rationalisation written after the fact. It does not: each entry names a measurement.

The setting. Everything runs against an LCF-style kernel implemented over Bourbaki's own formalism, described in a companion paper. Two of its properties matter here. The oracle is *exact*: a proposed witness costs at most a dead route, because the kernel judges, so the system can afford to guess. And the existential quantifier is not primitive — $(\exists x)R$ abbreviates a substitution of a τ-term into R — which is what later lets an organ *build* a witness out of a goal instead of searching for one.

Contributions.

1. **Failure as a datum** (Section 3). The need organ returns the obligations that would close the goal, judged by the kernel. 387 lines, protected by 20 tests.
2. **A growth law, and its evidence** (Section 4). Twenty-one organs, each traceable to a replayable failure. *The tool is deduced from the diagnosis.* The list is open, not a closed taxonomy.
3. **A frontier the machine drew** (Section 5). Chaining, then accepting proposals, then fabricating. The passage was forced by a gap the system wrote itself, and the improvement it brought is *not* quantitative: the same number of goals close, but the shape of the residual gap changes.
4. **A new algebra in three organs** (Section 6). Commutativity of $\oplus$ in under 2 s, associativity in 16.43 s, from two raw laws about $+$.
5. **Two exploitable failure signatures** (Section 7). Failing *by growing* indicates an upstream cause, not an insufficient budget; and a corpus of fourteen measured traps, including a structural law of the kernel.
6. **Our own drift, measured** (Section 8). The documentation of this work had gone stale on eight figures out of eight. We report it because a paper about naming what one lacks owes the reader the case where it stopped doing so.

What this paper does not claim. No new mathematics: the organs manipulate statements, and $\oplus$ is a toy whose two laws are immediate corollaries of those of $+$. No agent architecture: there is no planner, no learned policy, no training loop. And no generality: twenty-one organs were cut on two subjects, and nothing suggests the twenty-second problem will not demand five more — which is, in fact, what the thesis predicts.

2 Background: the pool, the route, the gap

We summarise only what is needed; the kernel and its trust boundary are the companion paper's subject.

A *theorem* is a triple: a finite set of hypotheses it still depends on, a conclusion, and the kernel rule that produced it. It is *closed* when the hypothesis set is empty. We write $\vdash R$ when the kernel can construct a closed theorem concluding R; every $\vdash$ below is witnessed by an object the repository can rebuild, never asserted.

A *goal* is a relation to be derived. A *pool* is a finite set of theorems available to the search. A *route* is one attempted derivation of the goal, and the *residue* of a route is the hypothesis set of the theorem it actually reached. A goal typically has several routes, and the same hypothesis may be open on one and closed on another — so every verdict below is stated of a hypothesis *on a route*.

Definition 1 (Gap)**.** *A* gap *of a route is a formula which, added to the pool, would let that route close its goal. The need organ returns, for a failed search, a finite set of gaps.*

The definition is deliberately weak: a gap is not claimed to be minimal, nor necessary, nor unique. It is what the search would have needed *on the route it took*. That modesty is what makes it computable and, as Section 5 shows, what makes its *shape* informative even when its size is not.

3 The need organ: failure with a shape

The organ takes a goal and a pool and returns either a kernel-certified proof, or a list of gaps. It is 387 lines of Python and is protected by 20 tests in `test_autonomie.py`; the whole discovery suite is **42 tests green in 32 min 50 s** (Section 9).

Two properties make the returned object useful rather than decorative.

The kernel judges, so guessing is cheap. Every candidate the organ tries is submitted to the kernel. A wrong guess produces no theorem and costs one dead route. This is what makes the proposer architecture of Section 5 affordable: the system may fabricate a term with no guarantee whatsoever, because nothing downstream will believe it on trust.

The gaps are the search's own words. They are not a rendering of the goal for human consumption; they are formulas the search would have consumed. That distinction is what makes them actionable — and it is what makes them a *measurement* of the distance remaining, rather than an estimate of it.

4 The catalogue, and its law of growth

Twenty-one organs have been identified. We write it that way, rather than "there are twenty-one", because the catalogue is a record of encounters and not a partition

proved exhaustive: an organ is added when a failure demands one, so the list grows by meeting new failures and nothing here bounds how many remain.

The claim. None of the twenty-one was designed in advance. Each answers a failure that can be replayed. Four examples, chosen because their diagnoses are of different kinds:

organ	what it does	**born from which diagnosis**
v4	instantiates the pool's ∀-facts on the goal	the induction hypothesis $S\{n\}$ sat *inert* in the pool: it was present and never used
v9	closes a goal $t = t$ by reflexivity	measured: a route was carrying $2k = 2k$ as a *gap*
v11	witnesses drawn from the free variables of pool facts	v10 extracted head terms only; a defined predicate unfolds and *buries* its argument
v18	instantiates a pool law at the moment of applying it	v17 matched left-hand sides literally: the pool said $a + b$, the goal contained $(a + b) + 1$ — the law was right, its *instance* was missing

Read the right-hand column and the left one becomes almost derivable. That asymmetry *is* the thesis: the tool is deduced from the diagnosis. An architecture designed up front would have produced a different table — one whose right column read "for generality" or "for completeness", which is what a rationalisation looks like.

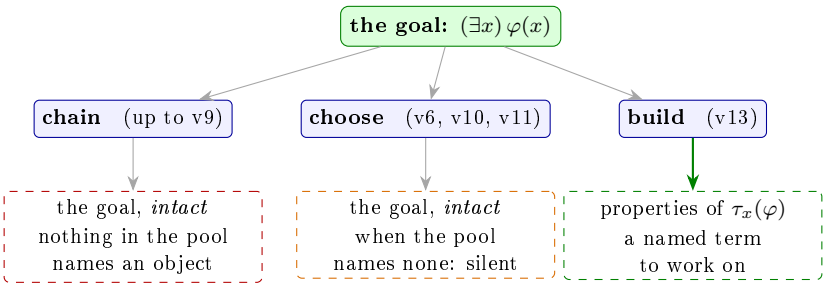
Remark 1 (an organ that vanished). The catalogue lists a v3, "merge the gaps of the direct route". Searching the code for it returns nothing: it was absorbed or removed, and the catalogue was never updated. We leave the observation here rather than quietly renumbering, because Section 8 is about exactly this.

5 Chaining, proposing, fabricating

The organ's history has three regimes, and the system passed between them under pressure rather than by design.

Chaining. Up to v9, the organ combines what it is given: it instantiates, detaches, decomposes, recomposes. Faced with an existential goal and a pool naming no suitable object, it reports the goal itself as its gap — correct, and useless.

Proposing. v6 turns the organ into an extension point: witness *proposers* are functions passed in, and `besoin.py` does not know what they are. v10 is the first



the passage was not designed: the terminal gap the system wrote was $\neg(n = n)$ *— read as a specification,* "give me a generator of witnesses"

Figure 1: The three regimes, and what each *returns when it fails* — the bottom row is the informative one. Chaining and choosing both hand back the goal untouched; building hands back properties of a named term. No more goals close (Remark 2): what changes is the *shape* of the residue. **This figure illustrates a distinction, it measures nothing** — the timings of the corresponding tests are in Section 9.

generic one — given $(\exists x)\varphi$, the candidates are the t of pool facts $t \in A$. It knows nothing about any particular problem.

Fabricating. v10 and v11 *choose* among objects the pool names. When the pool names none, they are silent. v13 *builds* the canonical witness $\tau_x(\varphi)$ from the goal alone, which Bourbaki's formalism makes a legitimate term rather than a trick.

What forced the passage. Not a design review. The system, searching a decomposition goal with only σ proposing $p := n$, wrote as its terminal gap

$$\neg(n = n).$$

A gap that cannot be filled by any pool: the search was asking for a contradiction because it had no way to name a different object. Read as a specification — and it is one, since gaps are formulas the search would consume — it says: *give me a generator of witnesses.* v6 and then v13 are the answer to that sentence.

Remark 2 (the improvement is not quantitative). v13 does not close more goals than v10. Judged on the number of gaps produced, it would have been discarded. What changes is their *shape*: on a decomposition goal with an empty pool, v10 returns the intact existential statement, and v13 returns properties of a named term. The system does not create information — it changes the form of the question. Measuring volume would have hidden the only thing that happened.

6 A freshly invented algebra, in three organs

The first fifteen organs were all cut on one conjecture. A different question was put on 11 August: is the machinery specific to that subject, or can it approach an algebra it has never seen?

The protocol is deliberately bare. Define an operation the repository does not know,

$$a \oplus b \;:=\; (a + b) + 1,$$

give the system *two raw laws* about $+$ — iterated associativity, commutativity — and nothing else, then ask for the properties of $\oplus$.

goal	closed by	measured
$a \oplus b = b \oplus a$	v16 (builds the congruence)	**1.75 s**
chained rewriting	v17	3.40 s
$(a \oplus b) \oplus c = a \oplus (b \oplus c)$	v17 + v18	**16.43 s**

Both results are closed with zero residual hypotheses. The durations are those of the corresponding tests, and therefore include imports and pool construction — they bound the cost from above, which is the direction that matters here.

The associativity chain is worth displaying, because it is what a literal matcher cannot find and what v18 makes reachable:

$$\begin{aligned}
((a + b) + 1) + c &= (a + b) + (1 + c) && \text{iterated associativity} \\
&= (a + b) + (c + 1) && \text{commutativity, } \textit{inside} \text{ the right operand} \\
&= a + (b + (c + 1)) && \text{associativity again} \\
&= a + ((b + c) + 1) && \text{and again, the other way}
\end{aligned}$$

Five steps, none of which matches a pool law *literally*: the pool says $a + b$, the goal contains $(a + b) + 1$. The law was right all along; what was missing was its instance at the subterm actually encountered. That is v18 in one sentence, and it was found by looking at a failure, not by reading a textbook on rewriting.

What this does not establish. $\oplus$ is a toy, and its two laws are immediate corollaries of those of $+$. Nothing mathematically new is produced. What is acquired is more modest and more solid: the system can reason equationally about a structure it has not seen, without being handed the instances. That is a prerequisite, not a result.

7 Two exploitable failure signatures

Failing by growing. The associativity goal above first failed in a way that looked like an exponential wall: cost doubling with each depth increment, and the number

of gaps frozen at one. The natural reading is "the problem is hard, raise the budget". It was wrong twice out of twice. The actual causes were upstream — an organ written twice over, and a bound set by judgement rather than by measurement: the search allowed three steps where the minimal chain, displayed above, takes five.

We state the resulting rule as a signature, because it is cheap to recognise:

> A search whose *cost explodes while its gap count stays fixed* is almost never repaired by more budget. Cost growing means the search is enumerating; gaps not moving means it is enumerating the wrong thing. Look upstream: at the exploration order, or at a bound nobody measured.

The practical consequence adopted since is to *measure the expected chain length before setting the bound*, which the relevant docstring now carries in figures rather than in prose.

Traps, paid for. The repository keeps fourteen traps that were met and paid for — not best practices copied from elsewhere. Three carry beyond this project:

- **The test that never ran.** A conclusion ("proposer v12 closes nothing") rested on a call that had silently fallen back to a default for want of a parameter. *Assert the capability before measuring it*; one line of introspection is enough. Looking for the reason uncovered the real defect — proposers were not crossing an intermediate layer at all.
- **The test both branches pass.** To show that fabricating beats choosing, the trial case was closed by both. A test that both branches pass discriminates nothing; the case where one fails has to be built on purpose.
- **Terms are opaque; formulas decompose.** In this kernel $N(7)$ and $N(3)+N(4)$ are both τ-terms with a single argument. There is nothing to descend into: no evaluator can work by walking a term. The only route is reconstruction — build the expected term and compare — which is practical because assemblies are hashable and equality is $O(1)$. Formulas, by contrast, decompose normally. The boundary is not a design choice; it is a fact about the substrate, and every tool of this kind has to be shaped by it.

Remark 3 (the same mistake twice in twenty minutes). The third trap was met once on terms, then immediately again on predicate recognition, where the code was navigating by `.sous[0].sous[1].sous[0]`. The second time, applying the law just written worked on the first attempt. The generalisation is blunter than the law: *never assume a structure — look at it.* Three lines of introspection settled in seconds what two successive assumptions had cost.

8 Our own drift, measured

This paper's material — a catalogue of the organs and a corpus of traps — was written on 10–12 August. Before drawing a single sentence from it, every figure it contained was re-measured. **Eight out of eight had drifted.**

the document says	measured 20 August	nature
15 tests in `test_autonomie.py`	**20**	tests added, document not
24 green, 40 min	**31** in the package; 42 with the corpus, **32 min 50**	and *faster* than stated
a catalogue of "nineteen lines"	**21** organs	the document contradicts
`besoin.py`, 259 lines	**387**	+128
organ v3	**no trace in the code**	absorbed or removed, neve
v16: 0.29 s	1.75 s	*not comparable, see below*
v18: 8 s	16.43 s	*idem*
v15: 102 s → 0 s	whole test in 2.87 s	*idem*

The reservation that matters. The last three lines do not show the document to be wrong. `pytest -durations` times the *whole test* — imports, pool construction, both passes — whereas the document's figures plainly timed *the organ call alone*. The two series measure different things, and writing "the document was wrong" would be as sloppy as the drift itself. What is established, and enough for Section 6, is an upper bound: under 2 s, 16.43 s.

The one claim we withdraw is v15's. The catalogue reports a first pass at 102 s and a second at 0 s — a striking ratio for a section on capitalisation. The whole test now runs in 2.87 s, so that ratio cannot be obtained in the current configuration. The compounding itself stands: the test asserts that the learned proposer alone, with no arithmetic access, recloses the goal. *Its magnitude does not.* We report the assertion and drop the number.

A figure we will not draw. The material's centrepiece was a curve of the gap count falling across successive probes, $14 \to 8 \to 6 \to 4 \to 1$, each drop labelled with the organ that caused it. It comes from a session journal and no script replays it. It is therefore **not in this paper**. A curve that cannot be reproduced has no place in an article whose thesis is that one measures instead of decreeing; opening on it would have been a refutation in Figure 1.

Why this section exists. It would have been easy to re-measure quietly, print the new figures, and say nothing. We report the drift because it is the paper's own thesis turned on the paper. The claim is that a system should name what it lacks; the documentation of that system had stopped doing so, and no test caught it — the kernel certifies derivations, not the prose written beside them. The same class of defect, at a different level, is the subject of the companion paper on fidelity.

9 Reproducibility

One command, one figure:

```
python -m pytest outils_ia/decouvertes/ tests/outils_ia/corpus/ -q
                            -durations=0
```

42 tests green in 32 min 50 s, exit 0, at the tag `preprint-v1`: 20 in `test_autonomie.py`, 9 in the autonomy sub-package, 2 on rediscovered lemmas, and 11 on the two most recent organs, which live in a different tree. Verdicts are read off the last line of the output file, never off a notification: three "exit 0" reports in this project turned out to be timeouts.

The cost is concentrated in one place. `test_euclide_infinitude` takes **1361.87 s** — **69% of the whole suite**; every organ test is under 17 s. Protecting twenty-one organs is cheap; proving Euclid is not.

10 Related work

Premise selection — the nearest neighbour, and the sharpest contrast. Given a goal and a large library, premise selection ranks which existing facts to feed the prover: by hand-built semantic features [7], by learned classifiers [1], by transformers over the library [10]. The question sounds like ours — *what should be added?* — and the difference is exactly where the interest lies. Premise selection *ranks what exists*; the need organ *names formulas that may not exist at all.* When our system reports $\neg(n = n)$ (Section 5), no ranking over any library could have produced that answer, because the answer is not a library element — it is a statement about what the search would have had to be given. Premise selection optimises retrieval; we characterise absence.

Proof repair. A second family takes a proof that *used to work* and mends it after its context changed, either by adapting automation to the change [14] or by learning from a corpus of repairs [12, 15]. The input is a broken artefact; ours is a search that never produced one. The two are complementary — repair presupposes a proof, and the last mile is precisely where there is none.

Failure as data. That discarded proof activity is worth capturing has been argued and instrumented [13], and current systems consume failures as training negatives or as repair prompts [4]. We share the premise and differ on the object. In those systems the recorded failure is an *unverified* artefact: a message, a trajectory, a model's own account of itself. Here it is a set of formulas the kernel will judge if anyone tries to use them. That is what makes a gap actionable rather than merely loggable.

Guiding search, versus describing its edge. Learned guidance — ENIGMA in a saturation prover [5], policies over tactic trees [16, 11], hypertree search [8] — aims at the same wall from the other side: it tries to make the search succeed more often. None of it says anything when the search still fails, which is the case this paper is about. The two are orthogonal, and a system with both would be strictly better than ours.

Exploration and conjecture. Theory exploration generates candidate lemmas and keeps those a prover confirms [6], and the recent survey catalogues what such systems propose [17]. Our organs generate *obligations* rather than conjectures: not interesting statements worth proving, but the specific formulas a stalled route needed. Systems that establish something by failing to prove — counterexample search [2], learned disproof [9] — form a third family, and refute rather than locate.

Growth by encounter. Finally, the shape of our result — a library of organs accreted from measured shortfalls, each earning its place — has a relative in program synthesis, where a system grows a library of abstractions from the problems it solves [3]. The mechanism differs (compression there, diagnosis here) but the epistemic claim is close, and we take it as evidence that the pattern is not an artefact of our setting.

Novelty, honestly stated. Formalised failure objects are not new; nor is premise selection, nor repair. What we have not found elsewhere is a search whose failure mode is *a kernel-checkable list of the formulas it would have consumed*, together with a catalogue showing that such objects, read as specifications, generated the tool's own extensions. As the companion papers insist, this claim is semi-decidable: no counterexample after methodical search, never more.

11 Limitations

Two subjects, twenty-one organs. The catalogue was cut on one conjecture and one toy algebra. Nothing here shows the organs generalise, and the thesis itself predicts that a genuinely new subject will demand new ones. The honest reading of "twenty-one" is as a record of what two problems required.

The catalogue is open. We report 21 identified organs, not a taxonomy proved exhaustive. Exhibiting a twenty-second is a finite act; proving there is none is not.

Gaps are route-relative and not minimal. Definition 1 claims nothing about minimality or necessity. A gap is what *the route taken* would have needed; a

different route may report a different set, and neither is canonical. The usefulness demonstrated here is practical, not theoretical.

Durations bound from above. All timings are test durations including setup, measured once on one Windows 11 machine (Python 3.13, pytest 9.0.3). They are upper bounds, not benchmarks, and no averaging over repetitions was done.

Withdrawn and unreproduced claims. v15's 102 s → 0 s is withdrawn (Section 8). Two further figures from the material — a factor of 100 on term evaluation, and a 24× ratio on the associativity search — were not re-run here; they are period measurements, and we do not present them as current.

One mind. Designed, operated and audited by one person. The re-measurement of Section 8 is a case where self-audit worked — but it worked because the material was about to be published, not because a process caught it.

12 Conclusion

The system reported here does not solve the problems it is pointed at. What it does is return, when it fails, a statement of what it would have needed — and that turned out to be enough to make the tool grow itself. Twenty-one organs, none designed in advance, each answering a failure that still replays; a frontier between chaining and fabricating that the machine crossed under the pressure of a gap it had written; and an algebra invented in the morning, reasoned about in the afternoon in under twenty seconds.

The two negative results are the ones we would keep. Failing *by growing* is a signature, and reading it correctly saved a chantier that more budget would not have opened. And the drift of our own documentation, eight figures out of eight, is the uncomfortable one: the discipline this paper recommends for a proof search is exactly the discipline its authors had stopped applying to their own notes.

A prover that says "no" teaches nothing. A prover that says "no, and here is the formula I lacked" hands back the only thing that makes the next attempt different from the last.

References

[1] Alexander A. Alemi, François Chollet, Niklas Een, Geoffrey Irving, Christian Szegedy, and Josef Urban. DeepMath — deep sequence models for premise selection. In *Proc. NeurIPS*, 2016.

[2] Jasmin C. Blanchette, Lukas Bulwahn, and Tobias Nipkow. Automatic proof and disproof in Isabelle/HOL. In *Proc. FroCoS*, 2011.

[3] Kevin Ellis, Catherine Wong, Maxwell Nye, et al. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proc. PLDI*, 2021.

[4] Emily First, Markus Rabe, Talia Ringer, and Yuriy Brun. Baldur: Whole-proof generation and repair with large language models. In *Proc. ESEC/FSE*, 2023.

[5] Jan Jakubův, Karel Chvalovský, Miroslav Olšák, Bartosz Piotrowski, Martin Suda, and Josef Urban. ENIGMA anonymous: Symbol-independent inference guiding machine. In *Proc. IJCAR*, 2020.

[6] Moa Johansson, Dan Rosén, Nicholas Smallbone, and Koen Claessen. Hipster: Integrating theory exploration in a proof assistant. In *Proc. CICM*, 2014.

[7] Cezary Kaliszyk, Josef Urban, and Jiří Vyskočil. Efficient semantic features for automated reasoning over large theories. In *Proc. IJCAI*, 2015.

[8] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving. In *Proc. NeurIPS*, 2022.

[9] Zenan Li, Zhaoyu Li, Kaiyu Yang, Xiaoxing Ma, and Zhendong Su. Learning to disprove: Formal counterexample generation with large language models. arXiv:2603.19514, 2026.

[10] Maciej Mikuła et al. Magnushammer: A transformer-based approach to premise selection. In *Proc. ICLR*, 2024.

[11] Stanislas Polu and Ilya Sutskever. Generative language modeling for automated theorem proving. arXiv:2009.03393, 2020.

[12] Tom Reichel, R. Henderson, Andrew Touchet, Andrew Gardner, and Talia Ringer. Proof repair infrastructure for supervised models: Building a large proof repair dataset. In *Proc. ITP*, 2023.

[13] Talia Ringer, Alex Sanchez-Stern, Dan Grossman, and Sorin Lerner. REPLica: REPL instrumentation for Coq analysis. In *Proc. CPP*, 2020.

[14] Talia Ringer, Nathaniel Yazdani, John Leo, and Dan Grossman. Adapting proof automation to adapt proofs. In *Proc. CPP*, 2018.

[15] Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, Jarod Alper, and Vasily Ilin. Learning to repair Lean proofs from compiler feedback. arXiv:2602.02990, 2026.

[16] Kaiyu Yang and Jia Deng. Learning to prove theorems via interacting with proof assistants. In *Proc. ICML*, 2019.

[17] Jian Zhang and Si-Cheng Tan. Automated conjecturing and theorem finding: A survey. *Journal of Computer Science and Technology*, 41(1):46–66, 2026. DOI 10.1007/s11390-026-6040-0.